# 1. Design Solution

## The system operates as follows:

1. Software timers periodically expire.
2. Their callbacks send task IDs into xGeneratorQueue.
3. The generator task receives the task ID, calculates the next absolute deadline, and calls release_dd_task().
4. The release request is sent through xMessageQueue to the DDS.
5. The DDS inserts the task into the active list and adjusts priorities.
6. The selected user task runs.
7. When the user task finishes, it calls complete_dd_task().
8. The DDS removes it from the active list and moves it to the completed list.
9. If a deadline is missed, the DDS moves the task to the overdue list and signals the generator through xGeneratorQueue.
10. The monitor task periodically requests all three lists and displays the current system state

# 2. F-Tasks & Their Priorities

## DD_Scheduler - Priority 4

**Usage**: Manages all deadline driven scheduling decisions. These decisions include: Task releases, completions, active/complete/overdue list updates, deadline miss detection, and priority reassignment.

**Priority Justification**: For the DDS designed for this project, the DD_Scheduler is the core task that must run before any other framework or user created task. If delayed, deadlines may be missed or task priorities may be assigned incorrectly.

## Generator Task - Priority 3

**Usage**: The generator task receives events from software timers and deadline-miss notifications. It then releases the corresponding user task with its next absolute deadline.

**Priority Justification**: The generator task must run soon after a timer has expired so that new jobs can be released. For this reason, it must have a high priority as it must run before user created tasks to ensure deadlines are set. However it must still remain below the DDS since the scheduler must process creation requests and update the system state first.

## Monitor Task - Priority 1

**Usage**: The monitoring task periodically requests the DDS for a list of the active, complete, and overdue tasks, then prints the stats to the console.

**Priority Justification**: The monitoring task is purely for visual purposes and debugging therefore is not a system critical component and must not mess with the other parts of the system.

# 3. DDS Interfacing Functions

## release_dd_task()

**Purpose:** The release_dd_task function is used to notify the DDS that a new task has been created and must be released and inserted into the active task list.

**Implementation:**

1. Package the received task data into a dd_task structure
2. Set the task handle, task type, task ID, and absolute deadline
3. Initialize the release time and completion time fields to zero
4. Package the message as type MSG_CREATE
5. Copy the created dd_task structure into the message
6. Send the message to the DDS through xMessageQueue
7. Wait for a confirmation reply from the DDS through xReplyQueue before continuing

**Operation:** When this function is called, the task information is sent to the DDS task. The DDS then records the actual release time, inserts the task into the active list in deadline order, adjusts task priorities, and sends a successful reply back to the caller.

## complete_dd_task()

**Purpose:** Complete_dd_task notifies the DDS that a task has completed and is ready to be deleted.

**Implementation:**

1. Package the message as type MSG_DELETE
2. Store the task_id of the completed task in the message
3. Send the message to the DDS through xMessageQueue
4. Wait for a confirmation reply from the DDS through xReplyQueue before continuing

**Operation:** When this function is called, the DDS receives the delete request and searches the active list for the task with the matching task ID. Once found, the task is removed from the active list, its completion time is recorded, and it is placed into the completed list. The DDS then updates priorities so the next earliest-deadline task can run.

## get_active_dd_task_list()

**Purpose:** The get_active_dd_task_list function requests the active task list from the DDS.

**Implementation:**

1. Package the message as type MSG_ACTIVE_LIST
2. Send the request message to the DDS through xMessageQueue
3. Wait for the DDS to return the pointer to the active task list through xListQueue
4. Return the received list pointer to the caller

**Operation:** When this function is called, the DDS receives the request and sends back the current pointer to the active task list which is checked to ensure the function was successful.

## get_completed_dd_task_list()

**Purpose:** The get_completed_dd_task_list function requests the complete task list from the DDS

.

**Implementation:**

1. Package the message as type MSG_COMPLETED_LIST
2. Send the request message to the DDS through xMessageQueue
3. Wait for the DDS to return the pointer to the completed task list through xListQueue
4. Return the received list pointer to the caller

**Operation:** When this function is called, the DDS receives the request and sends back the current pointer to the completed task list which is checked to ensure the function was successful.

## get_overdue_dd_task_list()

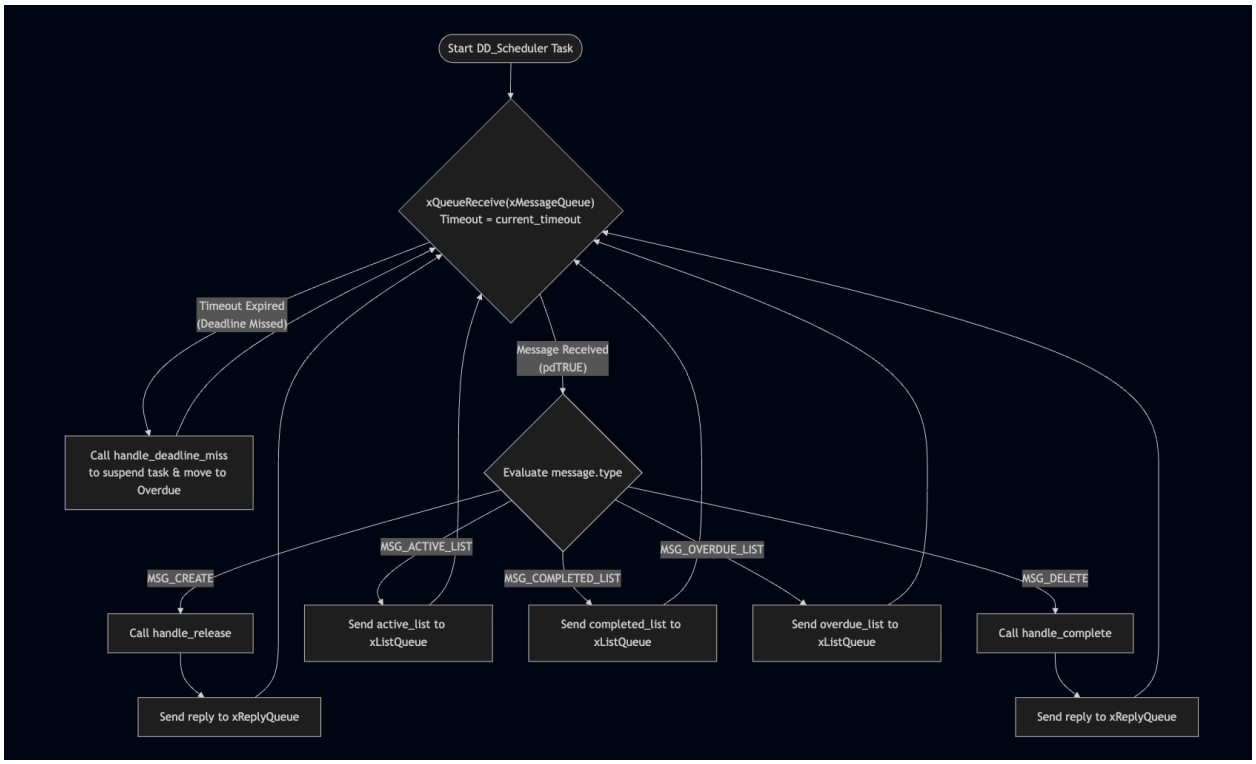
**Purpose:** The get_overdue_dd_task_list function requests the overdue task list from the DDS.

**Implementation:**

1. Package the message as type MSG_OVERDUE_LIST
2. Send the request message to the DDS through xMessageQueue
3. Wait for the DDS to return the pointer to the overdue task list through xListQueue
4. Return the received list pointer to the caller

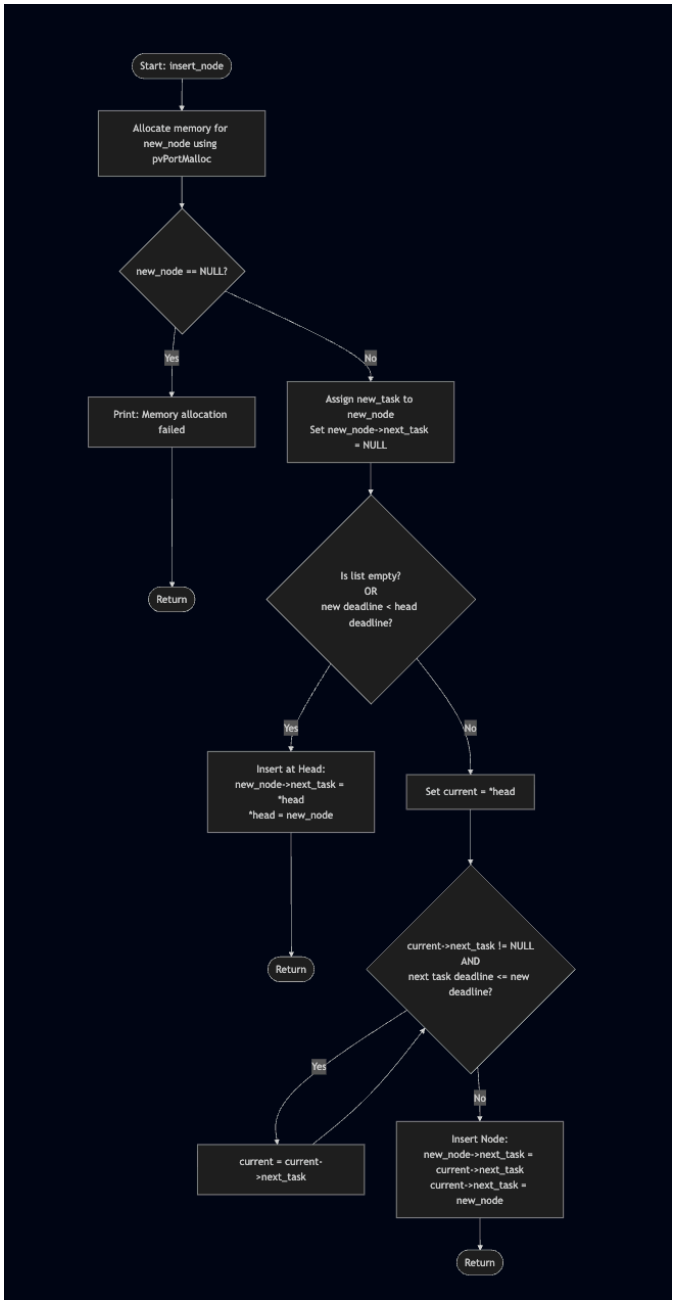
**Operation:** When this function is called, the DDS receives the request and sends back the current pointer to the overdue task list which is checked to ensure the function was successful.

# 4. System Flow Charts



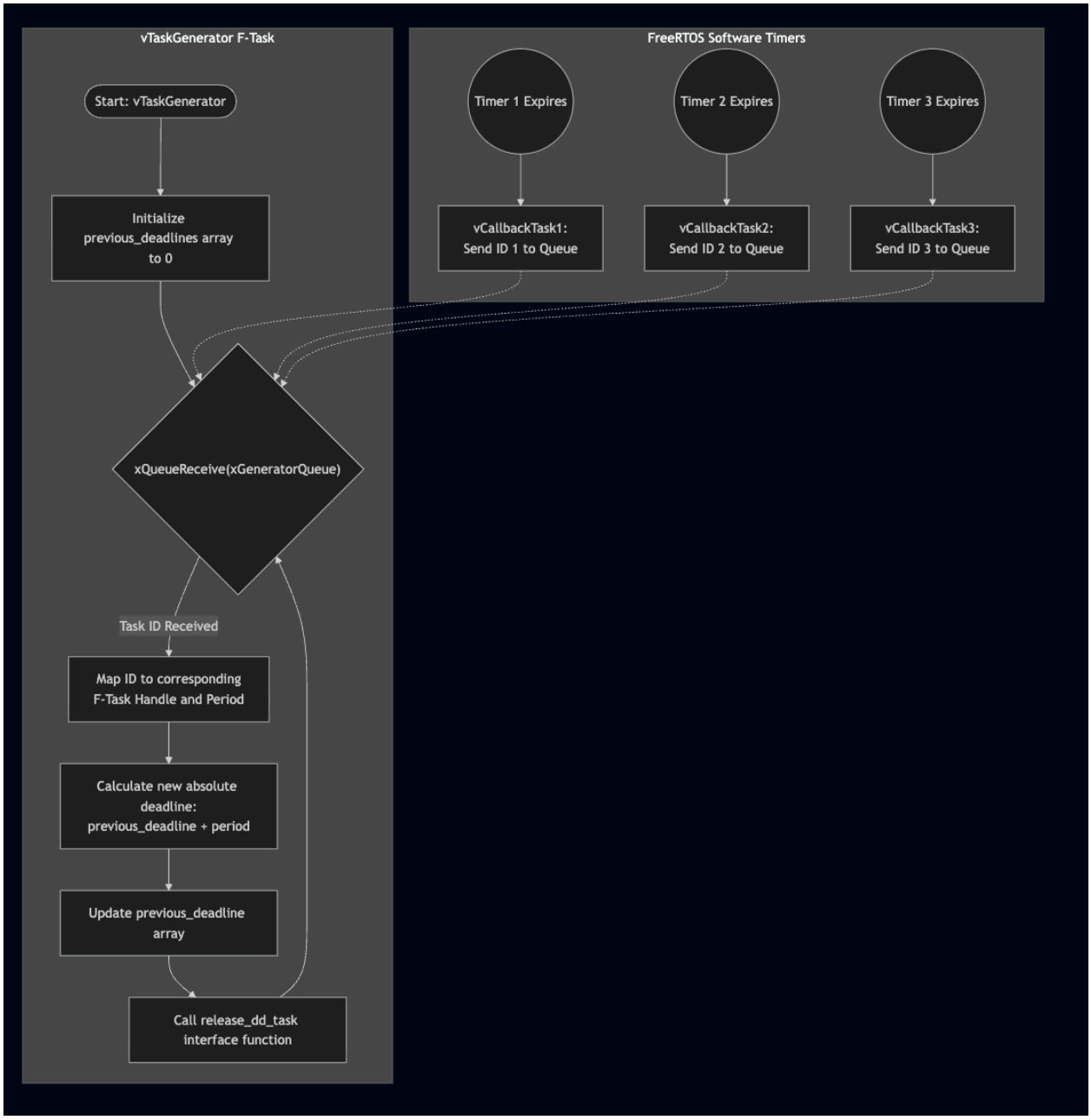

DD Scheduler Main Loop Flowchart

The Deadline-Driven Scheduler (DDS) runs as an infinite loop blocked on xQueueReceive. The cleverness of the EDF algorithm is that the queue's timeout parameter is strictly set to current_timeout (the exact time remaining until the active task's deadline). If the queue naturally times out, the scheduler immediately knows a deadline was missed and branches to handle_deadline_miss(). If a message arrives before the timeout, the scheduler evaluates the message type via a switch statement, executes the appropriate state-machine handler (handle_release, handle_complete, or sending list data), replies to the requesting task, and then returns to the blocked state to await the next event.




DD-Task List Sorting Algorithm (insert_node)

The scheduler maintains strict Earliest Deadline First (EDF) ordering using an insertion sort algorithm upon task release. When a new task arrives, the insert_node function allocates a new linked-list node via FreeRTOS's thread-safe pvPortMalloc. It first checks the base case: if the Active List is empty, or if the new task's absolute deadline is strictly less than the current head's deadline, it immediately pushes the new task to the front (head) of the list. Otherwise, it linearly traverses the list, comparing deadlines until it finds the correct chronological insertion point. This ensures that the task with the most urgent deadline is permanently stationed at the head of the Active List, providing O(1) lookup time for the scheduler's adjust_priorities() function.




DD-Task Generator Flowchart

The system utilizes a Single-Generator architecture to safely release periodic tasks without incurring race conditions. The generation cycle is driven by three FreeRTOS auto-reloading software timers corresponding to each task's period. When a timer expires, its callback uses an ISR-safe queue send function (xQueueSendFromISR) to push a simple task ID into the xGeneratorQueue. The vTaskGenerator F-Task, which runs at a high priority, spends most of its time blocked on this queue. When it receives a task ID, it maps the ID to its specific period and handle, calculates the new absolute deadline by adding the period to the task's previous deadline, and invokes the release_dd_task() interface function. The generator then immediately blocks again, waiting for the next timer or a deadline-miss re-release signal.

# 5. Discussion

(scheduler performance)

The performance of the custom Deadline-Driven Scheduler (DDS) was evaluated using three distinct scenarios: an under-subscribed system (Test Bench 1), an over-subscribed system (Test Bench 2), and a perfectly subscribed system (Test Bench 3). By analyzing the system's behavior across these hyper-periods, we can validate the Earliest Deadline First (EDF) algorithm and assess the real-world overhead introduced by the RTOS and our middleware.

**Test Bench #1: Under-Subscribed System (Guaranteed Feasibility)**

For Test Bench #1, the theoretical CPU utilization (U) is calculated as the sum of the execution times divided by their respective periods:

$$U = (95/500) + (150/500) + (250/750) = 0.19 + 0.30 + 0.33 = 0.823$$

Because the total utilization is ~82.3% ($U \leq 1.0$), the EDF algorithm mathematically guarantees that no deadlines will be missed. The measured results align perfectly with this theory; the Monitor Task reported 0 overdue tasks and 11 completed tasks by the end of the 1500ms hyper-period.

However, a comparison of the expected completion times versus the measured completion times reveals a consistent delay of exactly 2 milliseconds (e.g., Task 1 completed at 97ms instead of the theoretical 95ms; Task 2 at 247ms instead of 245ms). This variance represents the inherent system overhead. It is attributed to several RTOS factors:

1. Context Switching: The CPU requires time to suspend the worker tasks, save their states, and elevate the DDS task to PRIORITY_DDS to make scheduling decisions.
2. Queue Latency: Inter-task communication strictly utilizes FreeRTOS queues. Packaging, transmitting, and unpacking the dd_message structs introduces microsecond delays that accumulate.
3. List Sorting: Dynamic memory allocation (pvPortMalloc) and the O(n) traversal required to execute the insert_node() sorting algorithm take a finite amount of CPU cycles.

| # | Event | Expected (ms) | Measured (ms) |
|---|---|---|---|
| 1 | Task 1 Released | 0 | 2 |
| 2 | Task 2 Released | 0 | 2 |
| 3 | Task 3 Released | 0 | 2 |
| 4 | Task 1 Complete | 95 | 97 |
| 5 | Task 2 Complete | 245 | 247 |
| 6 | Task 3 Complete | 495 | 497 |

| | | | |
|---|---|---|---|
| 7 | Task 1 Released | 500 | 501 |
| 8 | Task 1 Complete | 595 | 596 |
| 9 | Task 2 Released | 500 | 501 |
| 10 | Task 2 Complete | 745 | 746 |
| 11 | Task 3 Released | 750 | 1002 |
| 12 | Task 1 Released | 1000 | 1002 |
| 13 | Task 1 Complete | 1095 | 1097 |
| 14 | Task 3 Complete | 1245 | 1247 |
| 15 | Task 2 Released | 1000 | 1002 |
| 16 | Task 2 Complete | 1395 | 1397 |
| 17 | Task 1 Released | 1500 | 1501 |

Table 1: Test Bench 1 Results (See Appendix B for Raw Result)

**Test Bench #2: Over-Subscribed System (Graceful Failure)**
For Test Bench #2, the execution period of Task 1 is halved to 250ms, drastically increasing the system load:

$$U = (95/250) + (150/500) + (250/750) = 0.38 + 0.30 + 0.33 = 1.013$$

With a utilization of ~101.3%, the system is mathematically guaranteed to fail. The performance objective here shifts from preventing a miss to gracefully handling one.

The system logs show that a deadline miss was successfully detected at t = 1500ms for Task ID: 1. Because the active task could not call the completion interface function before the DDS queue's current_timeout expired, the scheduler immediately interrupted the execution. The handle_deadline_miss() function successfully suspended the task, transferred it to the Overdue List, and signaled the Generator to attempt a re-release. The Monitor Task successfully recorded this, reporting 1 Overdue Task at the 2097ms mark, proving the scheduler can safely catch inevitabilities in overloaded states without crashing.

| Metric | Expected | Measured |
|---|---|---|
| **Number of Active DD-Tasks** | ---- | 0 |
| **Number of Complete DD-Tasks** | ---- | 11 |
| **Number of overdue DD-Tasks** | >= 1 | 0 |
| **Time Exclipsed** | 1500ms | 1998ms |

Table 2: Test Bench 2 Results (See Appendix C for Raw Result)

**Test Bench #3: 100% Utilization and Cascading Failure Analysis**

Test Bench 3 is mathematically designed to exactly max out the CPU:

$$U = (100/500) + (200/500) + (200/500) = 0.20 + 0.40 + 0.40 = 1.00 \ (100\%)$$

In a perfect mathematical model with zero overhead, this would be feasible. However, as established in Test Bench 1, the system has a practical overhead of ~2ms per task execution. Because the real-world utilization is technically 101.5%, the system accumulates a ~6ms execution deficit every 500ms hyper-period.

Initially, this deficit is hidden. After several seconds, the compounding deficit eventually exceeds a task's absolute deadline, causing the first overdue task to appear. At this point, a critical Double-Release race condition was observed, causing the overdue count to rapidly spike to 5 tasks and permanently cripple the scheduler.

When the deadline miss occurs, we believe two events happen simultaneously:

1. The DDS Timeout: The xQueueReceive in the DDS loop times out, triggering handle_deadline_miss(), which explicitly sends a signal to the Generator Queue to re-release the task.
2. The Hardware Timer: Because the tasks are periodic, the auto-reloading FreeRTOS software timer also expires at that exact absolute deadline, sending its own independent signal to the Generator Queue.

The Generator Queue processes both signals, calculating two new, staggered absolute deadlines and pushing the exact same physical FreeRTOS worker thread into the Active List twice. The DDS now contains two Linked-List nodes pointing to the same F-Task. As the system attempts to schedule these ghost tasks, they instantly time out because the physical task cannot complete two workloads simultaneously. This floods the Overdue List, causing the rapid spike to ~5 overdue tasks.

The DDS performance was highly deterministic and successfully maintained strict EDF scheduling. It correctly elevated the priorities of tasks with the nearest deadlines and reliably managed theoretical overloads. The 2ms overhead confirms that while the algorithm is theoretically sound, practical real-time embedded systems must leave a ~2-5% buffer in CPU utilization to account for RTOS and middleware operational jitter, and robust state-checking must be implemented to prevent compounding queue signals during overload conditions.

## Justification: Return Lists by Reference vs. Per-Task Generators

The Deadline-Driven Scheduler (DDS) returns the Active, Completed, and Overdue task lists to the Monitor Task by reference (passing memory pointers) rather than by value (performing a deep copy).

When the Monitor Task requests a system update via the get_active_dd_task_list() API (and its sister functions), the system must pass the list data through a FreeRTOS Queue. We analyzed two approaches for this transfer:

1. By Value (Deep Copy):
    - The DDS iterates through the requested list, allocates new memory (pvPortMalloc) for every single node, duplicates the data, and sends the head of this brand-new snapshot list to the Monitor.
    - This introduces an unacceptable amount of system overhead. Because the DDS operates at the highest system priority (PRIORITY_DDS = 4), forcing it to perform an O(n) deep-copy operation would block all other tasks from executing. As the Completed List grows over time (e.g., reaching dozens of nodes), the time taken to allocate memory and copy data would compound, introducing massive latency and causing active tasks to miss their hard deadlines. Furthermore, it would rapidly deplete the STM32F4's available SRAM, requiring the Monitor Task to manually free the memory after printing.
2. 2. By Reference (Pointer Transfer) - Chosen Implementation:
    - The DDS simply passes the 32-bit memory address (pointer) of the list's head node into the FreeRTOS queue (xListQueue).
    - This is an O(1) operation. Sending a single pointer through a queue takes almost zero CPU time and requires no dynamic memory allocation. The DDS can instantly reply to the Monitor Task and go back to sleeping/waiting for critical scheduling events, keeping the RTOS highly responsive.
    - The theoretical risk of passing by reference in a multi-threaded system is a race condition, the Monitor Task could be traversing the list while the DDS modifies it, potentially causing a crash if a node is deleted. However, our architecture safely mitigates this risk in two ways:
        i. The Monitor Task runs at the lowest priority (PRIORITY_LOW = 1). If a timer fires or a deadline misses while the Monitor is reading the list, the DDS (PRIORITY_DDS = 4) instantly preempts it, safely manipulates the pointers, and yields back.
        ii. In our state machine, DD-Task nodes are never actually destroyed (pvPortFree is never called during runtime). When a task finishes or misses a deadline, its node is safely unlinked using remove_node() and

appended to the Completed or Overdue lists. Therefore, the Monitor will never attempt to read a "dangling" or freed pointer.

Given the strict timing requirements of an Earliest Deadline First (EDF) scheduler, the massive performance and memory benefits of returning lists by reference far outweigh the mitigated concurrency risks.

## Justification: Single Generator vs. Per-Task Generator

The system utilizes a Single DD-Task Generator (vTaskGenerator) paired with individual FreeRTOS Software Timers, rather than spawning a dedicated generator F-Task for every individual DD-Task.

The lab manual requires periodic DD-Tasks to be released into the Active List at specific intervals (e.g., 250ms, 500ms, 750ms). We evaluated two primary architectures to accomplish this:

1. Per-Task Generators (Multiple Dedicated F-Tasks):
    - Create three separate F-Tasks (vGenerator1, vGenerator2, vGenerator3). Each runs in a continuous loop, calling vTaskDelayUntil() to wait for their specific period, calculating the new deadline, and calling release_dd_task().
    - * Memory Bloat: FreeRTOS requires a dedicated Stack (minimum 128 words, or 512 bytes on ARM Cortex-M4) and a Task Control Block (TCB) for every task created. Using three generators wastes a significant amount of SRAM on empty wait loops.
        i. The RTOS kernel would have to actively manage and context-switch between three additional high-priority tasks, stealing CPU cycles from the active user tasks.
        ii. In Test Bench 1, 2, and 3, all tasks release simultaneously at t = 0. Three independent generators waking up at the exact same tick could create race conditions or flood the DDS's xMessageQueue, causing unpredictable RTOS jitter.
2. Single Generator with Timer Callbacks - Chosen Implementation:
    - We created three auto-reloading FreeRTOS Software Timers corresponding to the periods of Task 1, 2, and 3. When a timer expires, its callback uses xQueueSendFromISR() to push a simple integer (task_id) into the xGeneratorQueue. A single F-Task (vTaskGenerator), running at PRIORITY_HIGH, waits on this queue. When it receives an ID, it looks up the period, calculates the absolute deadline, and signals the DDS.
    - The Advantages:

i. Only one stack and one TCB are required. Software timers are managed by the internal FreeRTOS daemon, costing negligible extra memory.
ii. The xGeneratorQueue acts as a natural serialization buffer. If all three timers fire at t=0, their IDs are simply lined up in the queue. The single generator processes them one by one, ensuring the DDS is fed orderly, thread-safe creation requests without race conditions.
iii. To add a 4th or 5th periodic task in the future, the system only requires an additional software timer and a new ID mapping in the generator's switch statement, rather than spinning up entirely new OS threads.

The Single Generator architecture significantly reduces RAM utilization and kernel overhead while providing a safer, serialized mechanism for concurrent task releases. By offloading the waiting mechanism to FreeRTOS Software Timers, the CPU is kept free to execute the deadline-sensitive active tasks.

## Justification: Pre-create Handles vs. Create/Delete per Period

The system pre-creates the User-Defined F-Task handles exactly once during system initialization (myDDSInit) and reuses them by suspending and resuming them, rather than dynamically creating and deleting the F-Tasks every time a DD-Task period occurs.

In a periodic Deadline-Driven system, tasks must be executed repeatedly at set intervals. We evaluated two approaches for managing the underlying FreeRTOS F-Tasks (the actual execution threads) that map to these DD-Tasks:

1. 1. Create/Delete Per Period (Dynamic Allocation):
    - When the Generator fires, it calls xTaskCreate() to spawn a brand new F-Task. When the task finishes its workload, it calls vTaskDelete(NULL) to destroy itself and free its memory.
    - The Drawbacks:
        i. Constantly allocating and freeing Task Control Blocks (TCBs) and memory stacks (at least 512 bytes per task) every few hundred milliseconds introduces severe memory fragmentation. While FreeRTOS's heap_4.c attempts to mitigate this by coalescing free blocks, the process is not perfectly deterministic and degrades system reliability over long uptimes.
        ii. xTaskCreate() and vTaskDelete() are computationally expensive kernel API calls. They require searching the heap for sufficient contiguous memory, initializing stack variables, and updating internal RTOS lists. Doing this for every single task release would massively inflate the ~2ms system overhead we measured, likely causing premature deadline misses in tightly subscribed scenarios like Test Bench 2.
        iii. If the system temporarily runs low on contiguous heap space during a spike in active tasks, xTaskCreate() will return

errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY, causing a catastrophic failure (triggering vApplicationMallocFailedHook).

2. 2. Pre-Create Handles and Suspend/Resume - Chosen Implementation:
    - All three User-Defined F-Tasks are instantiated once during myDDSInit() and immediately placed into a suspended state using vTaskSuspend(). They are written as infinite for(;;) loops. When the DDS elevates a task to the Active state, it simply calls vTaskResume() to wake it up. When the task finishes its work, it calls complete_dd_task() and then suspends itself (vTaskSuspend(NULL)), waiting for the next period.
    - The Advantages:
        i. Suspending and resuming tasks in FreeRTOS are lightweight, O(1) operations. They involve simply moving an existing TCB pointer between the Ready and Suspended state lists. This guarantees extremely fast, highly predictable context switching, which is mandatory for an Earliest Deadline First (EDF) scheduler.
        ii. 100% of the required memory for the worker tasks is allocated at startup. If the system boots successfully, it is mathematically guaranteed never to run out of memory for task stacks during runtime, completely eliminating dynamic allocation failures.
        iii. Because the physical thread never dies, local variables within the F-Task loop persist across periods if needed, and the DDS does not have to constantly track changing memory addresses for the TaskHandle_t.

For a hard real-time system, dynamic memory allocation during the critical scheduling loop is highly discouraged. Pre-creating the F-Tasks and utilizing the Suspend/Resume paradigm provides superior determinism, eliminates heap fragmentation, and significantly reduces the CPU overhead required to process periodic task releases.

# 6. Limitations & Possible Improvements

The current system, even though it was able to successfully meet the project requirements, does have several limitations to its scalability, efficiency, and robustness.

## Limitations

1. **Lack of Scalability:**
   A linked list sorted by deadlines is the main backbone of active task management for this project. While efficient for a small number of tasks, this does not scale well. As the number of tasks increase, the insertion, management, and removal of tasks becomes slower. This can lead to increased scheduling overhead and reduce the overall system responsiveness.

2. **Single Generator Bottleneck:**
   For the scale of this project, the single generator task is enough to handle all the task releases. However, as a system such as this begins to grow in complexity, this design becomes a bottleneck. Multiple release events must be handled sequentially which can introduce delays and reduce responsiveness if the system workload was increased.

3. **Minimal Fault Handling:**
   The system has limited mechanisms for handling errors and failures. For example:
   - Queue or memory allocation failures result in a halted system
   - No retry or recovery mechanisms meaning the system falls to fault and gets stuck.
   - No protection against unexpected runtime faults.

   Most of these issues were very prevalent during the testing for this project and the strengths of the fixes is unknown. Further investigation of these issues is required to better understand their causes.

4. **Expensive Overhead:**
   Several parts of the project introduce unnecessary overhead consume resources:
   - Frequency queue communications between tasks,
   - Linked list operations for scheduling,
   - Continuous monitoring output using printf,
   - User-tasks additionally use wait loops which consume CPU resources.

## Possible Improvements

1. **Replace Link List:**
   The active list can be replaced with a more efficient structure such as a priority queue. [1] This would reduce insertion time, improving overall system responsiveness.

2. **Improve Scalability of Generator:**

To improve the scalability of the generator, a multiple task generator system could be used, or a more efficient signaling mechanism such as a direct-to-task notification system [2].

3. **Improve Fault Tolerance:**
   To improve fault tolerance and increase system robustness:
   - Add retry mechanisms for queue operations if they fail,
   - Handle memory allocation failures,
   - Integrated watchdog timers for systems recovery [3].

4. **Reduce Monitoring Task Overhead:**
   The monitor task currently performs frequent reporting, which adds an unnecessary load. While easily the monitoring task could be removed as it is not system critical, other improves could be:
   - Reducing reporting frequency,
   - Trigger updates based on events,
   - Replace printf with a lighter logging mechanism.

# 7. Summary

This project successfully designed and implemented a custom Deadline-Driven Scheduler (DDS) utilizing the Earliest Deadline First (EDF) algorithm layered on top of the native FreeRTOS kernel. By dynamically elevating the priorities of User-Defined F-Tasks based on absolute deadlines, the system strictly adhered to EDF scheduling principles.

Testing across three mathematical benchmarks validated the scheduler's performance. In an under-subscribed state (Test Bench 1), the DDS guaranteed all deadlines and maintained 100% feasibility. In an over-subscribed state (Test Bench 2), the system demonstrated graceful degradation, successfully catching inevitable deadline misses and securely moving them to an Overdue List without crashing the RTOS. Finally, analysis of the system under 100% theoretical utilization (Test Bench 3) revealed an inherent RTOS operational overhead of approximately 2 milliseconds per task, confirming that real-world embedded systems require a marginal CPU buffer to maintain strict real-time feasibility. Overall, the architecture safely and deterministically managed deadline-sensitive workloads while avoiding global variables through strict queue-based inter-task communication.

# References:

[1] GeeksforGeeks, “C program to implement priority queue,” GeeksforGeeks, https://www.geeksforgeeks.org/c/c-program-to-implement-priority-queue/

[2] FreeRTOS.org, “FreeRTOS direct to task notifications - freertosTM,” FreeRTOS, https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/03-Direct-to-task-notifications/01-Task-notifications

[3] UnTraDe 3 et al., “Implementing a watchdog timer in C++11,” Stack Overflow, https://stackoverflow.com/questions/35824878/implementing-a-watchdog-timer-in-c11

# Appendix A:

```
/* Standard includes. */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "stm32f4_discovery.h"
#include "stm32f4xx_adc.h"
#include "stm32f4xx_gpio.h"

/* Kernel includes. */
#include "stm32f4xx.h"
#include "../FreeRTOS_Source/include/FreeRTOS.h"
#include "../FreeRTOS_Source/include/queue.h"
#include "../FreeRTOS_Source/include/semphr.h"
#include "../FreeRTOS_Source/include/task.h"
#include "../FreeRTOS_Source/include/timers.h"

static void prvSetupHardware(void);
void myDDSInit(void);

/*-----------------------------------------------------------------*/
/*---------------------------- DEFINES ---------------------------- */
#define mainQUEUE_LENGTH 10

#define YES         1
#define NO          0

#define PRIORITY_DDS        4  // Highest priority for the scheduler itself
#define PRIORITY_HIGH       3  // For the Generator
#define PRIORITY_MED                2  // For the EDF Head task
#define PRIORITY_LOW        1  // For all other active tasks

// Let's create handles for all 3 tasks and the generator
TaskHandle_t xUserTask1;
TaskHandle_t xUserTask2;
TaskHandle_t xUserTask3;
TaskHandle_t xGeneratorTask;
TaskHandle_t xMonitorTask;

TimerHandle_t xTimerUser1;
TimerHandle_t xTimerUser2;
TimerHandle_t xTimerUser3;
```

```
/*---------------------------- TEST BENCHES ---------------------------- */
// Change this to TB_2 or TB_3 to test the other scenarios
#define TB_2

#ifdef TB_1
   #define T1_EXEC   pdMS_TO_TICKS(95)
   #define T1_PERIOD pdMS_TO_TICKS(500)
   #define T2_EXEC   pdMS_TO_TICKS(150)
   #define T2_PERIOD pdMS_TO_TICKS(500)
   #define T3_EXEC   pdMS_TO_TICKS(250)
   #define T3_PERIOD pdMS_TO_TICKS(750)
#elif defined(TB_2)
   #define T1_EXEC   pdMS_TO_TICKS(95)
   #define T1_PERIOD pdMS_TO_TICKS(250)
   #define T2_EXEC   pdMS_TO_TICKS(150)
   #define T2_PERIOD pdMS_TO_TICKS(500)
   #define T3_EXEC   pdMS_TO_TICKS(250)
   #define T3_PERIOD pdMS_TO_TICKS(750)
#elif defined(TB_3)
   #define T1_EXEC   pdMS_TO_TICKS(100)
   #define T1_PERIOD pdMS_TO_TICKS(500)
   #define T2_EXEC   pdMS_TO_TICKS(200)
   #define T2_PERIOD pdMS_TO_TICKS(500)
   #define T3_EXEC   pdMS_TO_TICKS(200)
   #define T3_PERIOD pdMS_TO_TICKS(500)
#endif


/*---------------------------- Structures ---------------------------- */

typedef enum {PERIODIC, APERIODIC} task_type;

typedef struct {
   TaskHandle_t t_handle;
   task_type type;
   uint32_t task_id;
   uint32_t release_time;
   uint32_t absolute_deadline;
   uint32_t completion_time;
} dd_task;

typedef struct dd_task_list {
   dd_task task;
```

```
    struct dd_task_list *next_task;
} dd_task_list;

typedef enum {
    MSG_CREATE,
    MSG_DELETE,
    MSG_ACTIVE_LIST,
    MSG_COMPLETED_LIST,
    MSG_OVERDUE_LIST
} message_type;

typedef struct {
    message_type type;
    dd_task task;
    uint32_t task_id; // Used for DELETE messages
} dd_message;

/*---------------------------- Function Prototypes ---------------------------- */
void insert_node(dd_task_list **head, dd_task new_task);
dd_task_list* remove_node(dd_task_list **head, uint32_t task_id);
uint32_t get_list_size(dd_task_list *head);

void adjust_priorities(void);
void handle_release(dd_task new_task);
void handle_complete(uint32_t task_id);
void handle_deadline_miss(void);

void release_dd_task(TaskHandle_t t_handle, task_type type, uint32_t task_id, uint32_t
absolute_deadline);
void complete_dd_task(uint32_t task_id);
dd_task_list* get_active_dd_task_list(void);
dd_task_list* get_completed_dd_task_list(void);
dd_task_list* get_overdue_dd_task_list(void);

void DD_Scheduler(void *pvParameters);
void vUserTask1(void *pvParameters);
void vUserTask2(void *pvParameters);
void vUserTask3(void *pvParameters);
void vTaskGenerator(void *pvParameters);
void vCallbackTask1(TimerHandle_t xTimer);
void vCallbackTask2(TimerHandle_t xTimer);
void vCallbackTask3(TimerHandle_t xTimer);
void vMonitorTask(void *pvParameters);
```

```
/*---------------------------- FreeRTOS Queues ---------------------------- */
QueueHandle_t xMessageQueue;
QueueHandle_t xReplyQueue;
QueueHandle_t xListQueue;
QueueHandle_t xGeneratorQueue;

/*---------------------------- Global Variables ---------------------------- */
// Global pointers for our 3 lists
dd_task_list *active_list = NULL;
dd_task_list *completed_list = NULL;
dd_task_list *overdue_list = NULL;

// Global timeout tracker for the queue
TickType_t current_timeout = portMAX_DELAY;

/*---------------------------- MAIN ---------------------------- */

int main(void)
{
    // 1. Initialize hardware (clocks, etc.)
    prvSetupHardware();

    // 2. Initialize DDS Queues, Tasks, and Timers
    myDDSInit();

    // 3. Start the RTOS scheduler
    vTaskStartScheduler();


    // The program should never reach here
    for (;;);
    return 0;
}

/*---------------------------- Initialization ---------------------------- */

void myDDSInit(void) {
    // 1. Create Queues
    xMessageQueue = xQueueCreate(mainQUEUE_LENGTH, sizeof(dd_message));
    xReplyQueue   = xQueueCreate(mainQUEUE_LENGTH, sizeof(uint32_t));
    xListQueue    = xQueueCreate(mainQUEUE_LENGTH, sizeof(dd_task_list*));
    xGeneratorQueue = xQueueCreate(mainQUEUE_LENGTH, sizeof(uint32_t));
```

```
    if (xMessageQueue == NULL || xReplyQueue == NULL || xListQueue == NULL ||
xGeneratorQueue == NULL) {
        printf("Queue Creation Failed!\n");
        for (;;);
    }

    // 2. Create DDS Task (Highest Priority)
    xTaskCreate(DD_Scheduler, "DDS", 256, NULL, PRIORITY_DDS, NULL);

    // 3. Create User Tasks (Start Suspended)
    xTaskCreate(vUserTask1, "T1", configMINIMAL_STACK_SIZE, NULL, PRIORITY_LOW,
&xUserTask1);
    vTaskSuspend(xUserTask1);
    xTaskCreate(vUserTask2, "T2", configMINIMAL_STACK_SIZE, NULL, PRIORITY_LOW,
&xUserTask2);
    vTaskSuspend(xUserTask2);
    xTaskCreate(vUserTask3, "T3", configMINIMAL_STACK_SIZE, NULL, PRIORITY_LOW,
&xUserTask3);
    vTaskSuspend(xUserTask3);

    // 4. Create Generator Task
    xTaskCreate(vTaskGenerator, "GEN", configMINIMAL_STACK_SIZE, NULL,
PRIORITY_HIGH, &xGeneratorTask);

    // 5. Create and Start the Timers
    xTimerUser1 = xTimerCreate("TMR1", T1_PERIOD, pdTRUE, 0, vCallbackTask1);
    xTimerUser2 = xTimerCreate("TMR2", T2_PERIOD, pdTRUE, 0, vCallbackTask2);
    xTimerUser3 = xTimerCreate("TMR3", T3_PERIOD, pdTRUE, 0, vCallbackTask3);

    xTimerStart(xTimerUser1, 0);
    xTimerStart(xTimerUser2, 0);
    xTimerStart(xTimerUser3, 0);

    // 6. Jump-start the generator so all 3 tasks release at t=0
    uint32_t id1 = 1, id2 = 2, id3 = 3;
    xQueueSend(xGeneratorQueue, &id1, 0);
    xQueueSend(xGeneratorQueue, &id2, 0);
    xQueueSend(xGeneratorQueue, &id3, 0);

    // 7. Create Monitor Task
    xTaskCreate(vMonitorTask, "MON", configMINIMAL_STACK_SIZE, NULL, PRIORITY_LOW,
&xMonitorTask);
}
```

```
/*---------------------------- List Helper Functions ---------------------------- */

void insert_node(dd_task_list **head, dd_task new_task){
    // Using pvPortMalloc for heap_4.c compatibility
    dd_task_list *new_node = (dd_task_list *)pvPortMalloc(sizeof(dd_task_list));

    if (new_node == NULL) {
        printf("Memory allocation failed\n");
        return;
    }

    new_node->task = new_task;
    new_node->next_task = NULL;

    // Case 1: Empty list OR new task has the earliest deadline (new head)
    if (*head == NULL || (*head)->task.absolute_deadline > new_task.absolute_deadline) {
        new_node->next_task = *head;
        *head = new_node;
        return;
    }

    // Case 2: Insert in the middle or at the tail
    dd_task_list *current = *head;
    while (current->next_task != NULL &&
           current->next_task->task.absolute_deadline <= new_task.absolute_deadline) {
        current = current->next_task;
    }

    new_node->next_task = current->next_task;
    current->next_task = new_node;
}

dd_task_list* remove_node(dd_task_list **head, uint32_t task_id){
    if (*head == NULL) return NULL; // Empty list

    dd_task_list *current = *head;
    dd_task_list *prev = NULL;

    // Search for the node
    while (current != NULL && current->task.task_id != task_id) {
        prev = current;
        current = current->next_task;
    }
```

```
    // Target not found
    if (current == NULL) return NULL;

    // Case 1: Removing the head
    if (prev == NULL) {
        *head = current->next_task;
    }
    // Case 2: Removing from the middle or tail
    else {
        prev->next_task = current->next_task;
    }

    current->next_task = NULL; // Sever the link to the old list
    return current;
}

uint32_t get_list_size(dd_task_list *head){
    uint32_t count = 0;
    dd_task_list *current = head;

    while (current != NULL) {
        count++;
        current = current->next_task;
    }
    return count;
}


/*---------------------------- DDS State Machine Functions ---------------------------- */

/* adjust_priorities() — set head of Active List to HIGH, others to LOW; recalculate timeout */
void adjust_priorities(void) {
    if (active_list == NULL) {
        current_timeout = portMAX_DELAY; // Nothing to do, wait forever
        return;
    }

    // 1. Set the head task to MED priority AND wake it up!
    if (active_list->task.t_handle != NULL) {
        vTaskPrioritySet(active_list->task.t_handle, PRIORITY_MED);
        vTaskResume(active_list->task.t_handle); // <--- ADD THIS EXACT LINE
    }
```

```
    // 2. Demote all other active tasks to LOW priority
    dd_task_list *curr = active_list->next_task;
    while (curr != NULL) {
        if (curr->task.t_handle != NULL) {
            vTaskPrioritySet(curr->task.t_handle, PRIORITY_LOW);
        }
        curr = curr->next_task;
    }

    // 3. Recalculate queue timeout based on the head task's deadline
    TickType_t current_time = xTaskGetTickCount();
    if (active_list->task.absolute_deadline > current_time) {
        current_timeout = active_list->task.absolute_deadline - current_time;
    } else {
        current_timeout = 0; // Deadline has already passed
    }
}

/* handle_release() — assign release time, insert into Active List, adjust priorities */
void handle_release(dd_task new_task) {
    new_task.release_time = xTaskGetTickCount();
    insert_node(&active_list, new_task);
    adjust_priorities();
}

/* handle_complete() — assign completion time, move from Active to Completed, adjust
priorities */
void handle_complete(uint32_t task_id) {
    dd_task_list *completed_node = remove_node(&active_list, task_id);

    if (completed_node != NULL) {
        completed_node->task.completion_time = xTaskGetTickCount();

        // Push to the head of the completed list (order doesn't matter here)
        completed_node->next_task = completed_list;
        completed_list = completed_node;
    }
    adjust_priorities();
}

/* handle_deadline_miss() — move head of Active to Overdue, signal Generator */
void handle_deadline_miss(void) {
    if (active_list == NULL) return;
```

```
    // The head is the one that missed the deadline
    dd_task_list *missed_node = active_list;
    active_list = active_list->next_task; // Remove from active

    // Suspend the F-Task so it stops consuming CPU time
    if (missed_node->task.t_handle != NULL) {
        vTaskSuspend(missed_node->task.t_handle);
    }

    // Move to overdue list
    missed_node->next_task = overdue_list;
    overdue_list = missed_node;

    // Output for testing/checkpoint
    printf("\n>>> DEADLINE MISSED for Task ID: %u <<<\n", missed_node->task.task_id);

    // FIX: Signal the Generator to re-release the task via the Queue
    uint32_t missed_id = missed_node->task.task_id;
    xQueueSend(xGeneratorQueue, &missed_id, portMAX_DELAY);

    adjust_priorities(); // Setup for the next task in line
}


/*---------------------------- Interface Functions ---------------------------- */

void release_dd_task(TaskHandle_t t_handle, task_type type, uint32_t task_id, uint32_t
absolute_deadline) {
    uint32_t reply;

    // Package the task data
    dd_task new_task = {
        .t_handle = t_handle,
        .type = type,
        .task_id = task_id,
        .absolute_deadline = absolute_deadline,
        .release_time = 0,     // Set by DDS
        .completion_time = 0   // Set by DDS
    };

    // Package the message
    dd_message message = { .type = MSG_CREATE, .task = new_task };

    // Send to DDS and wait for SUCCESS reply
```

```
    xQueueSend(xMessageQueue, &message, portMAX_DELAY);
    xQueueReceive(xReplyQueue, &reply, portMAX_DELAY);
}

void complete_dd_task(uint32_t task_id) {
    uint32_t reply;
    dd_message message = { .type = MSG_DELETE, .task_id = task_id };

    xQueueSend(xMessageQueue, &message, portMAX_DELAY);
    xQueueReceive(xReplyQueue, &reply, portMAX_DELAY);
}

dd_task_list* get_active_dd_task_list(void) {
    dd_task_list *list_copy;
    dd_message message = { .type = MSG_ACTIVE_LIST };

    xQueueSend(xMessageQueue, &message, portMAX_DELAY);
    xQueueReceive(xListQueue, &list_copy, portMAX_DELAY);
    return list_copy;
}

dd_task_list* get_completed_dd_task_list(void) {
    dd_task_list *list_copy;
    dd_message message = { .type = MSG_COMPLETED_LIST };

    xQueueSend(xMessageQueue, &message, portMAX_DELAY);
    xQueueReceive(xListQueue, &list_copy, portMAX_DELAY);
    return list_copy;
}

dd_task_list* get_overdue_dd_task_list(void) {
    dd_task_list *list_copy;
    dd_message message = { .type = MSG_OVERDUE_LIST };

    xQueueSend(xMessageQueue, &message, portMAX_DELAY);
    xQueueReceive(xListQueue, &list_copy, portMAX_DELAY);
    return list_copy;
}

/*---------------------------- Tasks ---------------------------- */

/* DD_Scheduler task — main loop: receive message -> handle it -> block again */
void DD_Scheduler(void *pvParameters) {
    dd_message message_received;
```

```
    uint32_t reply_status = YES;

    for (;;) {
        // Block until a message arrives OR the deadline timeout expires
        if (xQueueReceive(xMessageQueue, &message_received, current_timeout) == pdTRUE) {

            // We received a message before a deadline fired
            switch (message_received.type) {
                case MSG_CREATE:
                    handle_release(message_received.task);
                    xQueueSend(xReplyQueue, &reply_status, portMAX_DELAY);
                    break;

                case MSG_DELETE:
                    handle_complete(message_received.task_id);
                    xQueueSend(xReplyQueue, &reply_status, portMAX_DELAY);
                    break;

                case MSG_ACTIVE_LIST:
                    xQueueSend(xListQueue, &active_list, portMAX_DELAY);
                    break;

                case MSG_COMPLETED_LIST:
                    xQueueSend(xListQueue, &completed_list, portMAX_DELAY);
                    break;

                case MSG_OVERDUE_LIST:
                    xQueueSend(xListQueue, &overdue_list, portMAX_DELAY);
                    break;
            }
        }
        else {
            // The queue timed out! A deadline has been missed.
            handle_deadline_miss();
        }
    }
}

/*---------------------------- User Tasks ---------------------------- */
void vUserTask1(void *pvParameters) {
    TickType_t currentTick, executeTick, prevTick;
    for (;;) {
        executeTick = T1_EXEC;
        currentTick = xTaskGetTickCount();
```

```
        while (executeTick > 0) {
            prevTick = currentTick;
            currentTick = xTaskGetTickCount();
            if (currentTick != prevTick) executeTick--;
        }
        printf("Task 1 Completed at %u\n", xTaskGetTickCount());
        complete_dd_task(1);
        vTaskSuspend(NULL);
    }
}

void vUserTask2(void *pvParameters) {
    TickType_t currentTick, executeTick, prevTick;
    for (;;) {
        executeTick = T2_EXEC;
        currentTick = xTaskGetTickCount();
        while (executeTick > 0) {
            prevTick = currentTick;
            currentTick = xTaskGetTickCount();
            if (currentTick != prevTick) executeTick--;
        }
        printf("Task 2 Completed at %u\n", xTaskGetTickCount());
        complete_dd_task(2);
        vTaskSuspend(NULL);
    }
}

void vUserTask3(void *pvParameters) {
    TickType_t currentTick, executeTick, prevTick;
    for (;;) {
        executeTick = T3_EXEC;
        currentTick = xTaskGetTickCount();
        while (executeTick > 0) {
            prevTick = currentTick;
            currentTick = xTaskGetTickCount();
            if (currentTick != prevTick) executeTick--;
        }
        printf("Task 3 Completed at %u\n", xTaskGetTickCount());
        complete_dd_task(3);
        vTaskSuspend(NULL);
    }
}

/*---------------------------- Task Generator ---------------------------- */
```

```
// Timers feed the Generator Queue
void vCallbackTask1(TimerHandle_t xTimer) { uint32_t id = 1;
xQueueSendFromISR(xGeneratorQueue, &id, NULL); }
void vCallbackTask2(TimerHandle_t xTimer) { uint32_t id = 2;
xQueueSendFromISR(xGeneratorQueue, &id, NULL); }
void vCallbackTask3(TimerHandle_t xTimer) { uint32_t id = 3;
xQueueSendFromISR(xGeneratorQueue, &id, NULL); }

void vTaskGenerator(void *pvParameters) {
   uint32_t current_task_id;
   // Arrays to track the previous deadlines to calculate the new absolute deadline
   uint32_t previous_deadlines[3] = {0, 0, 0};

   for (;;) {
      // Block until a timer OR a deadline miss tells us to release a task
      if (xQueueReceive(xGeneratorQueue, &current_task_id, portMAX_DELAY) == pdTRUE) {

         uint32_t period = 0;
         TaskHandle_t handle = NULL;

         // Map ID to its parameters
         if (current_task_id == 1)      { period = T1_PERIOD; handle = xUserTask1; }
         else if (current_task_id == 2) { period = T2_PERIOD; handle = xUserTask2; }
         else if (current_task_id == 3) { period = T3_PERIOD; handle = xUserTask3; }

         // Calculate new absolute deadline: previous deadline + relative deadline
         previous_deadlines[current_task_id - 1] += period;
         uint32_t absolute_deadline = previous_deadlines[current_task_id - 1];

         printf("Generator: Releasing Task %u (Deadline: %u)\n", current_task_id,
absolute_deadline);

         // Call the interface function
         release_dd_task(handle, PERIODIC, current_task_id, absolute_deadline);
      }
   }
}

/*---------------------------- Monitor Task ---------------------------- */

void vMonitorTask(void *pvParameters) {
   for (;;) {
      // Wake up every 500ms to check the system state
```

```
        vTaskDelay(pdMS_TO_TICKS(500));

        // 1. Request the list pointers from the DDS
        dd_task_list *active = get_active_dd_task_list();
        dd_task_list *completed = get_completed_dd_task_list();
        dd_task_list *overdue = get_overdue_dd_task_list();

        // 2. Count the nodes in each list
        uint32_t active_count = get_list_size(active);
        uint32_t completed_count = get_list_size(completed);
        uint32_t overdue_count = get_list_size(overdue);

        // 3. Print the report
        printf("\n=====================================\n");
        printf(" Monitor Report at %u ms\n", xTaskGetTickCount());
        printf("   Active Tasks:    %u\n", active_count);
        printf("   Completed Tasks: %u\n", completed_count);
        printf("   Overdue Tasks:   %u\n", overdue_count);
        printf("=====================================\n\n");
    }
}


/*---------------------------- FreeRTOS Hooks & Setup ---------------------------- */

void vApplicationMallocFailedHook( void )
{
   /* Called if a call to pvPortMalloc() fails because there is insufficient
   free memory available in the FreeRTOS heap. */
   for( ;; );
}

void vApplicationStackOverflowHook( TaskHandle_t pxTask, signed char *pcTaskName )
{
   ( void ) pcTaskName;
   ( void ) pxTask;
   for( ;; );
}

void vApplicationIdleHook( void )
{
   volatile size_t xFreeStackSpace;
   xFreeStackSpace = xPortGetFreeHeapSize();
   if( xFreeStackSpace > 100 )
```

```
  {
      // Memory is sufficient
  }
}

static void prvSetupHardware( void )
{
  /* Ensure all priority bits are assigned as preemption priority bits. */
  NVIC_SetPriorityGrouping( 0 );
  /* TODO: Setup the clocks, etc. here, if they were not configured before main() was called. */
}
```

# Appendix B:

Testbed 1:

```
Generator: Releasing Task 1 (Deadline: 500)
Generator: Releasing Task 2 (Deadline: 500)
Generator: Releasing Task 3 (Deadline: 750)
Task 1 Completed at 97
Task 2 Completed at 247
Task 3 Completed at 497
Generator: Releasing Task 1 (Deadline: 1000)
Generator: Releasing Task 2 (Deadline: 1000)
Task 1 Completed at 596
Task 2 Completed at 746
Generator: Releasing Task 3 (Deadline: 1500)
Generator: Releasing Task 1 (Deadline: 1500)
Generator: Releasing Task 2 (Deadline: 1500)
Task 1 Completed at 1097
Task 3 Completed at 1247
Task 2 Completed at 1397

Generator: Releasing Task 3 (Deadline: 2250)
Generator: Releasing Task 1 (Deadline: 2000)
Generator: Releasing Task 2 (Deadline: 2000)
Task 1 Completed at 1597
Task 2 Completed at 1747
Task 3 Completed at 1997

========================================
 Monitor Report at 1998 ms
   Active Tasks:    0
   Completed Tasks: 11
   Overdue Tasks: 0
========================================
```

# Appendix C:

```
Generator: Releasing Task 1 (Deadline: 250)
Generator: Releasing Task 2 (Deadline: 500)
Generator: Releasing Task 3 (Deadline: 750)
Task 1 Completed at 97
Task 2 Completed at 247
Generator: Releasing Task 1 (Deadline: 500)
Task 1 Completed at 345
Generator: Releasing Task 2 (Deadline: 1000)
Generator: Releasing Task 1 (Deadline: 750)
Task 3 Completed at 593
Task 1 Completed at 688
Generator: Releasing Task 3 (Deadline: 1500)
Generator: Releasing Task 1 (Deadline: 1000)
Task 2 Completed at 839
Task 1 Completed at 934
Generator: Releasing Task 2 (Deadline: 1500)
Generator: Releasing Task 1 (Deadline: 1250)
Task 1 Completed at 1096
Generator: Releasing Task 1 (Deadline: 1500)
Task 3 Completed at 1280
Task 2 Completed at 1430

>>> DEADLINE MISSED for Task ID: 1 <<<
Generator: Releasing Task 1 (Deadline: 1750)
Generator: Releasing Task 3 (Deadline: 2250)
Generator: Releasing Task 2 (Deadline: 2000)
Generator: Releasing Task 1 (Deadline: 2000)
Task 1 Completed at 1529
Task 2 Completed at 1679
Generator: Releasing Task 1 (Deadline: 2250)
Task 1 Completed at 1845
Generator: Releasing Task 2 (Deadline: 2500)
Generator: Releasing Task 1 (Deadline: 2500)
Task 3 Completed at 2096

=====================================
 Monitor Report at 2097 ms
   Active Tasks:    3
   Completed Tasks: 14
   Overdue Tasks:   1
=====================================
```